

SQL Practice — Bank Management System

AGGREGATE FUNCTIONS with WHERE, GROUP BY, HAVING, ORDER BY

Name: Hansika

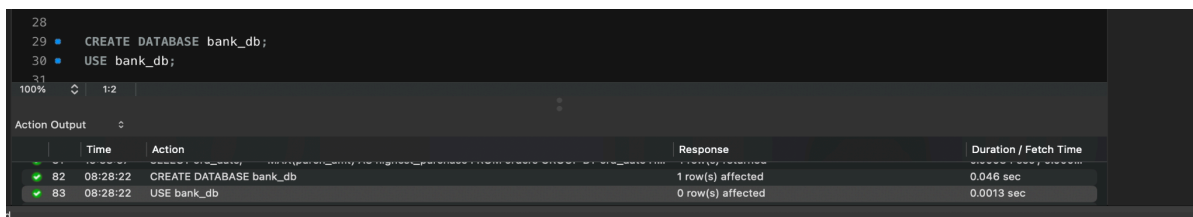
Roll No: 2520030237

Section: 7

Email: gunapatihansikareddy@klh.edu.in

Step 1 — Create Database

```
CREATE DATABASE bank_db;  
USE bank_db;
```

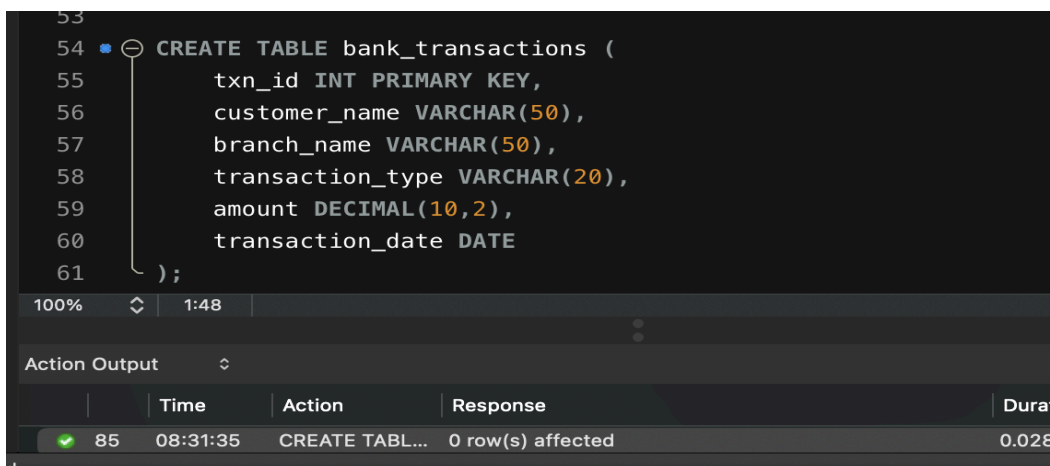


A screenshot of a SQL IDE showing the execution of two commands. The first command, 'CREATE DATABASE bank_db;', is highlighted in blue. The second command, 'USE bank_db;', is also highlighted in blue. Below the commands, the 'Action Output' table shows the results of the execution.

	Time	Action	Response	Duration / Fetch Time
82	08:28:22	CREATE DATABASE bank_db	1 row(s) affected	0.046 sec
83	08:28:22	USE bank_db	0 row(s) affected	0.0013 sec

Step 2 — Create Bank_Transactions Table (DDL)

```
CREATE TABLE bank_transactions (  
    txn_id INT PRIMARY KEY,  
    customer_name VARCHAR(50),  
    branch_name VARCHAR(50),  
    transaction_type VARCHAR(20),  
    amount DECIMAL(10,2),  
    transaction_date DATE  
);
```



A screenshot of a SQL IDE showing the execution of a DDL statement. The statement is highlighted in blue. Below the statement, the 'Action Output' table shows the results of the execution.

	Time	Action	Response	Duration / Fetch Time
85	08:31:35	CREATE TABL...	0 row(s) affected	0.028

Step 3 — Insert Data and Display

```
INSERT INTO bank_transactions VALUES
(101,'Hansika','Hyderabad','Deposit',5000,'2024-01-05'),
(102,'Sita','Hyderabad','Withdrawal',2000,'2024-01-06'),
(103,'Kiran','Vijayawada','Deposit',12000,'2024-01-08'),
(104,'Anil','Vizag','Deposit',8000,'2024-01-10'),
(105,'Priya','Hyderabad','Withdrawal',3500,'2024-01-11'),
(106,'Ramesh','Vizag','Deposit',15000,'2024-01-12'),
(107,'Keerthi','Vijayawada','Withdrawal',1000,'2024-01-13'),
(108,'Rahul','Hyderabad','Deposit',9000,'2024-01-14'),
(109,'Sneha','Vizag','Withdrawal',4000,'2024-01-15'),
(110,'Madhu','Vijayawada','Deposit',11000,'2024-01-16');

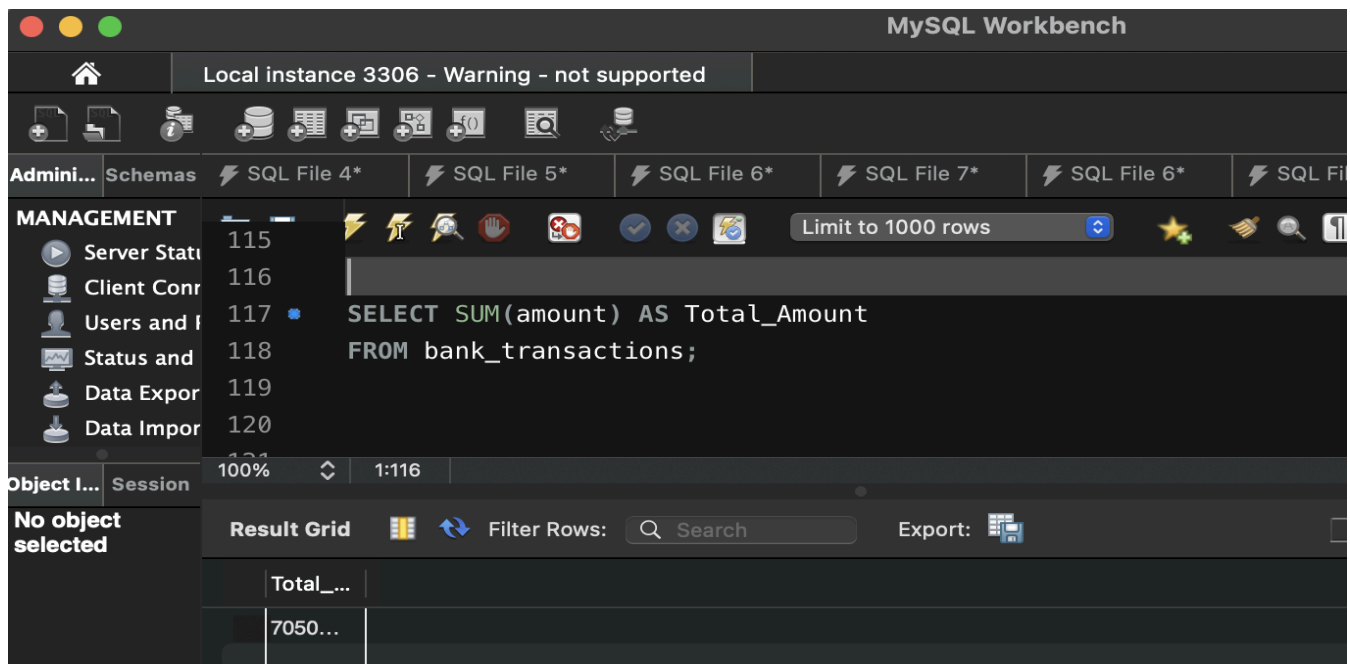
SELECT * FROM bank_transactions;
```

The screenshot shows a database management tool interface. The top bar indicates 'Local instance 3306 - Warning - not supported'. The left sidebar has a 'MANAGEMENT' section with icons for Server Status, Client Connections, Users and Roles, Status and Settings, Data Export, Data Import, Startup / Stop, and Server Logs. Below this is an 'Object Explorer' showing 'No object selected'. The main area displays SQL code being executed, with line numbers 84 through 98. The code is the same as in Step 3. Below the code, a 'Result Grid' shows the output of the SELECT statement. The grid has 7 columns: txn_id, customer_name, branch_name, transaction_type, amount, and transaction_date. It contains 11 rows of data, corresponding to the transactions inserted in the previous step. The last row shows NULL values for all columns.

txn_id	customer_name	branch_name	transaction_ty...	amount	transaction_d...
101	Hansika	Hyderabad	Deposit	5000.00	2024-01-05
102	Sita	Hyderabad	Withdrawal	2000.00	2024-01-06
103	Kiran	Vijayawada	Deposit	12000.00	2024-01-08
104	Anil	Vizag	Deposit	8000.00	2024-01-10
105	Priya	Hyderabad	Withdrawal	3500.00	2024-01-11
106	Ramesh	Vizag	Deposit	15000.00	2024-01-12
107	Keerthi	Vijayawada	Withdrawal	1000.00	2024-01-13
108	Rahul	Hyderabad	Deposit	9000.00	2024-01-14
109	Sneha	Vizag	Withdrawal	4000.00	2024-01-15
110	Madhu	Vijayawada	Deposit	11000.00	2024-01-16
NULL	NULL	NULL	NULL	NULL	NULL

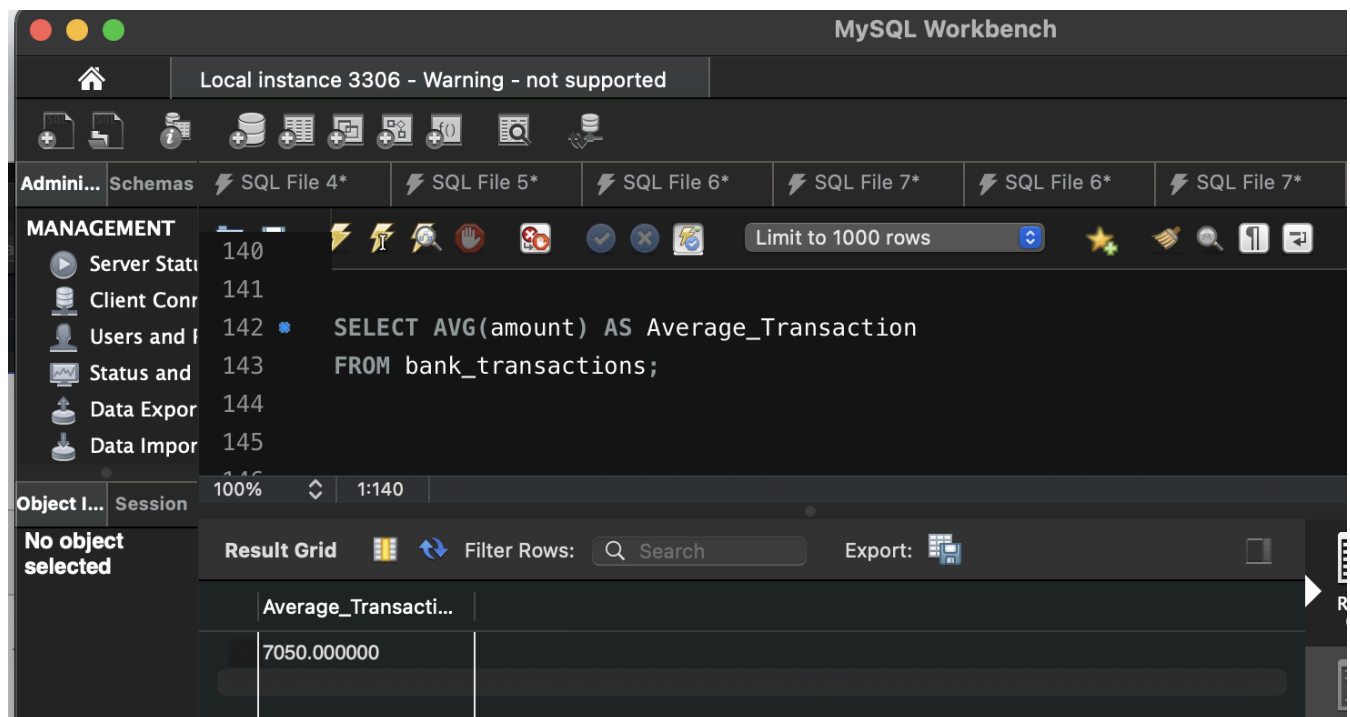
Step 4 — Question 1: SUM()

```
SELECT SUM(amount) AS Total_Amount
FROM bank_transactions;
```



Step 5 — Question 2: AVG()

```
SELECT AVG(amount) AS Average_Transaction
FROM bank_transactions;
```



Step 6 — Question 3: MAX()

```
SELECT MAX(amount) AS Highest_Transaction
FROM bank_transactions;
```

The screenshot shows the MySQL Workbench interface. The SQL editor contains the query: `SELECT MAX(amount) AS Highest_Transaction FROM bank_transactions;`. The query is executed, and the result is displayed in the 'Result Grid' tab. The result shows a single row with the value 15000.00 for the column 'Highest_Transaction'.

Highest_Transaction
15000.00

Step 7 — Question 4: MIN()

Scenario: The audit department wants to know the smallest transaction amount recorded.

```
SELECT MIN(amount) AS Lowest_Transaction
FROM bank_transactions;
```

The screenshot shows the MySQL Workbench interface. The SQL editor contains the query: `SELECT MIN(amount) AS Lowest_Transaction FROM bank_transactions;`. The query is executed, and the result is displayed in the 'Result Grid' tab. The result shows a single row with the value 1000.00 for the column 'Lowest_Transaction'.

Lowest_Transaction
1000.00

Step 8 — Question 5: COUNT()

```
SELECT COUNT(*) AS Total_Transactions
FROM bank_transactions;
```

The screenshot shows the MySQL Workbench interface. The SQL editor contains the query: `SELECT COUNT(*) AS Total_Transactions FROM bank_transactions;`. The query is executed, and the result is displayed in the 'Result Grid' at the bottom. The grid shows a single row with the column 'Total_Transactions' and the value '10'.

Total_Transactions
10

Step 9 — Question 6: WHERE + SUM()

```
SELECT SUM(amount) AS Total_Deposit
FROM bank_transactions
WHERE transaction_type='Deposit';
```

The screenshot shows the MySQL Workbench interface. The SQL editor contains the query: `SELECT SUM(amount) AS Total_Deposit FROM bank_transactions WHERE transaction_type='Deposit';`. The query is executed, and the result is displayed in the 'Result Grid' at the bottom. The grid shows a single row with the column 'Total...' and the value '6000...'.

Total...
6000...

Step 10 — Question 7: GROUP BY

```
SELECT branch_name,  
       SUM(amount) AS Total_Amount  
FROM bank_transactions  
GROUP BY branch_name;
```

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
SELECT branch_name,  
       SUM(amount) AS Total_Amount  
FROM bank_transactions  
GROUP BY branch_name;
```

The query is executed, and the results are displayed in the Result Grid. The grid shows three rows of data:

branch_name	Total_Amount
Hyderabad	1950...
Vijayawada	2400...
Vizag	2700...

Step 11 — Question 8: GROUP BY + HAVING

```
SELECT branch_name,  
       SUM(amount) AS Total_Amount  
FROM bank_transactions  
GROUP BY branch_name  
HAVING SUM(amount) > 20000;
```

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

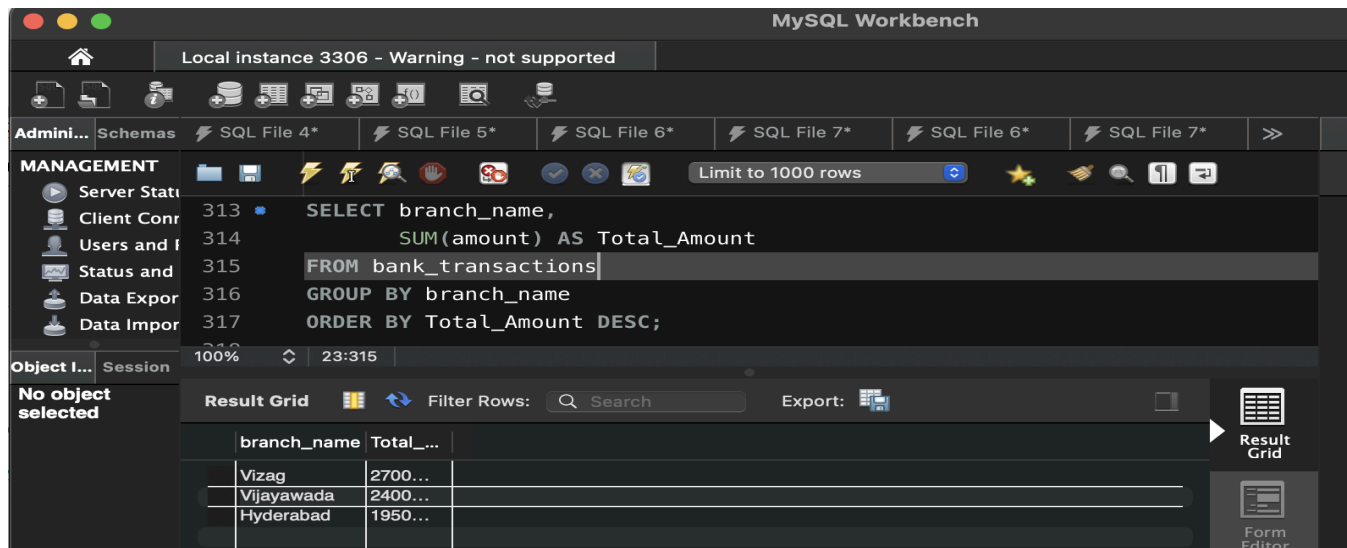
```
SELECT branch_name,  
       SUM(amount) AS Total_Amount  
FROM bank_transactions  
GROUP BY branch_name  
HAVING SUM(amount) > 20000;
```

The query is executed, and the results are displayed in the Result Grid. The grid shows two rows of data:

branch_name	Total_Amount
Vijayawada	2400...
Vizag	2700...

Step 12 — Question 9: GROUP BY + ORDER BY

```
SELECT branch_name,  
       SUM(amount) AS Total_Amount  
FROM bank_transactions  
GROUP BY branch_name  
ORDER BY Total_Amount DESC;
```



MySQL Workbench interface showing the execution of a SQL query. The query is:

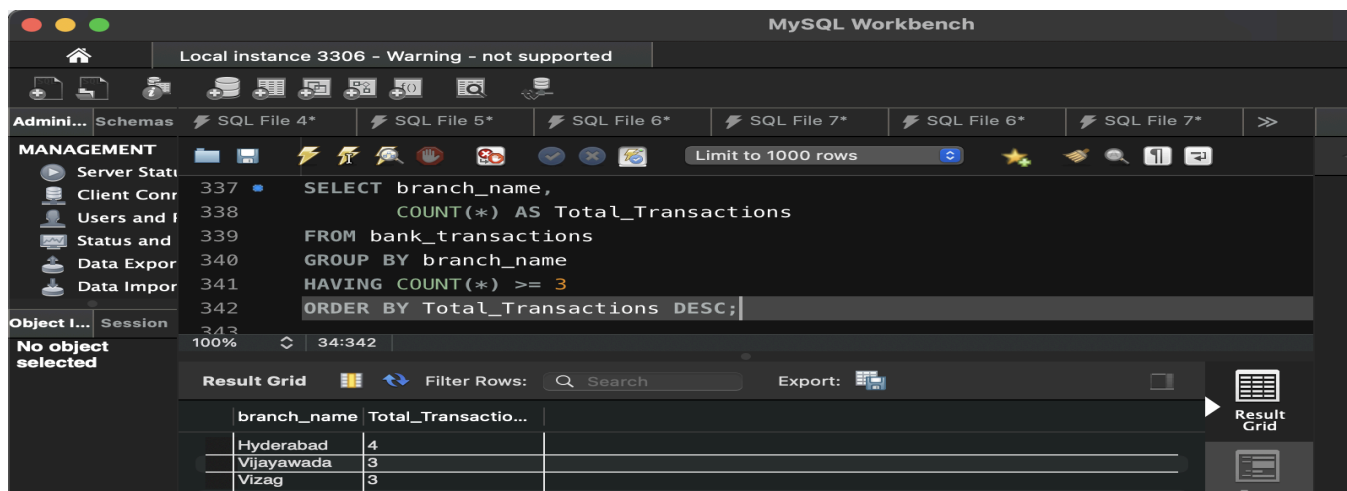
```
SELECT branch_name,  
       SUM(amount) AS Total_Amount  
FROM bank_transactions  
GROUP BY branch_name  
ORDER BY Total_Amount DESC;
```

The result grid displays the following data:

branch_name	Total_Amount
Vizag	2700...
Vijayawada	2400...
Hyderabad	1950...

Step 13 — Question 10: GROUP BY + HAVING + ORDER BY

```
SELECT branch_name,  
       COUNT(*) AS Total_Transactions  
FROM bank_transactions  
GROUP BY branch_name  
HAVING COUNT(*) >= 3  
ORDER BY Total_Transactions DESC;
```



MySQL Workbench interface showing the execution of a SQL query. The query is:

```
SELECT branch_name,  
       COUNT(*) AS Total_Transactions  
FROM bank_transactions  
GROUP BY branch_name  
HAVING COUNT(*) >= 3  
ORDER BY Total_Transactions DESC;
```

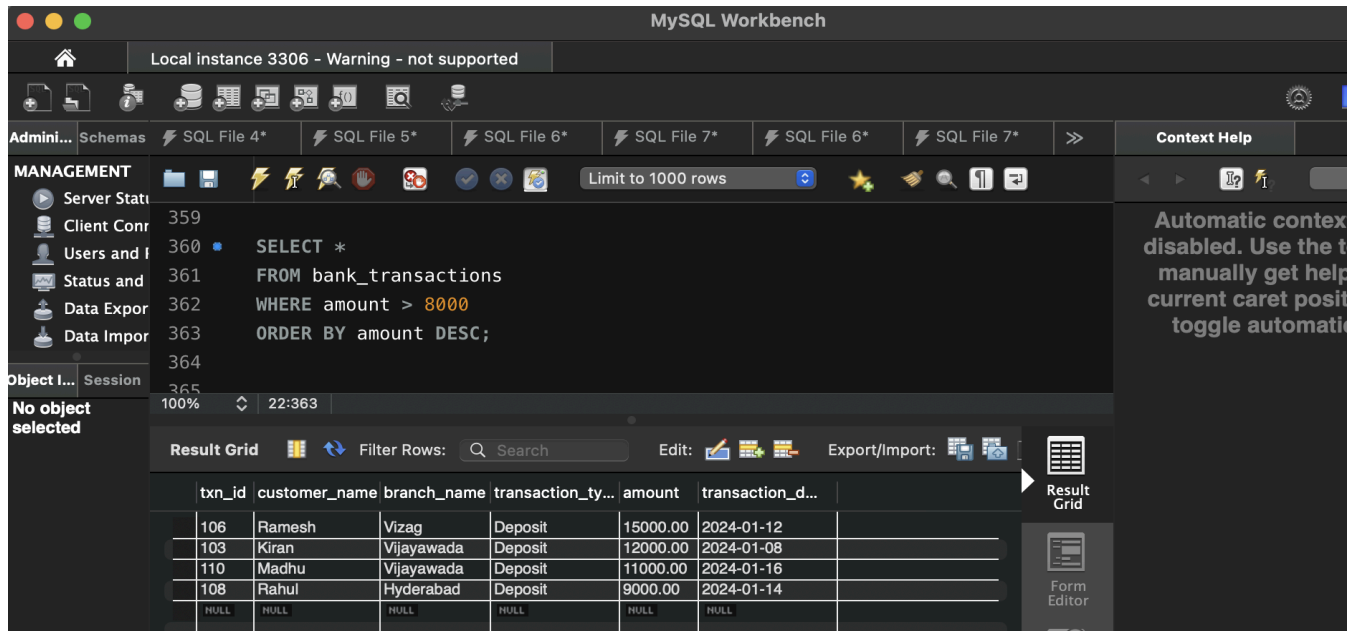
The result grid displays the following data:

branch_name	Total_Transactions
Hyderabad	4
Vijayawada	3
Vizag	3

Step 14 — Question 11: WHERE + ORDER BY

Scenario: The fraud detection team wants all transactions greater than \$8,000.

```
SELECT *
FROM bank_transactions
WHERE amount > 8000
ORDER BY amount DESC;
```



MySQL Workbench interface showing the execution of a SQL query. The query is:

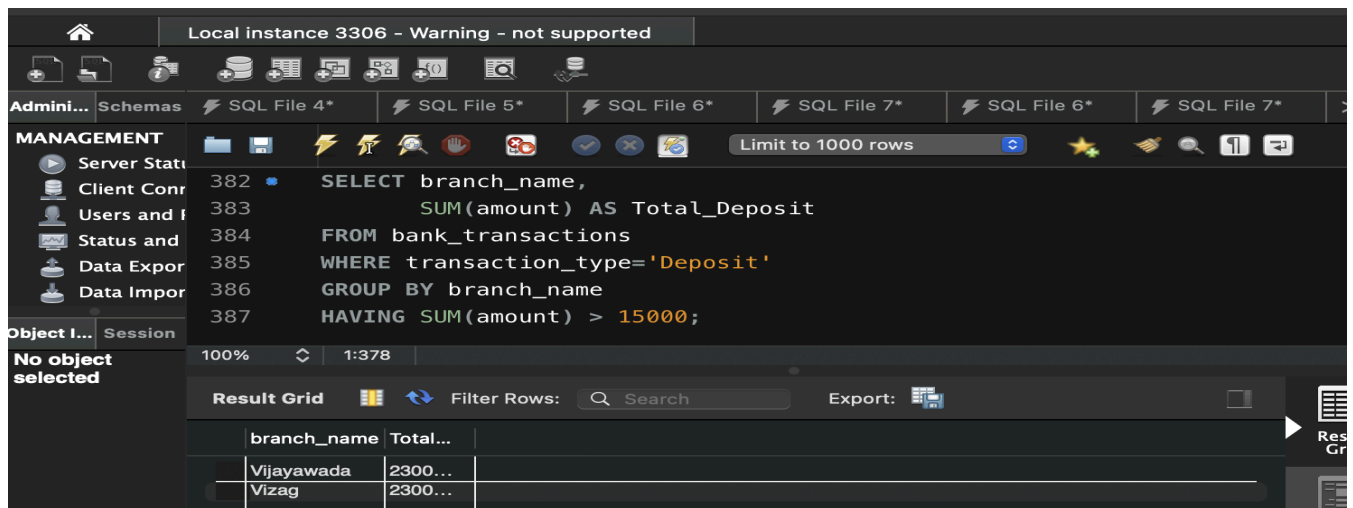
```
SELECT *
FROM bank_transactions
WHERE amount > 8000
ORDER BY amount DESC;
```

The result grid displays the following data:

txn_id	customer_name	branch_name	transaction_ty...	amount	transaction_d...
106	Ramesh	Vizag	Deposit	15000.00	2024-01-12
103	Kiran	Vijayawada	Deposit	12000.00	2024-01-08
110	Madhu	Vijayawada	Deposit	11000.00	2024-01-16
108	Rahul	Hyderabad	Deposit	9000.00	2024-01-14
NULL	NULL	NULL	NULL	NULL	NULL

Step 15 — Question 12: WHERE + GROUP BY + HAVING

```
SELECT branch_name,
       SUM(amount) AS Total_Deposit
FROM bank_transactions
WHERE transaction_type='Deposit'
GROUP BY branch_name
HAVING SUM(amount) > 15000;
```



MySQL Workbench interface showing the execution of a SQL query. The query is:

```
SELECT branch_name,
       SUM(amount) AS Total_Deposit
FROM bank_transactions
WHERE transaction_type='Deposit'
GROUP BY branch_name
HAVING SUM(amount) > 15000;
```

The result grid displays the following data:

branch_name	Total...
Vijayawada	2300...
Vizag	2300...